WARNING We'll rely on a specific solution that ensures each user can only access their own orders via the `/orders` endpoint. However, this won't prevent developers from using the other methods exposed by `OrderRepository` in the future and leaking private data. If you'd like to know how to improve this solution, refer to chapter 17 of *Spring Security in Action* by Laurențiu Spilcă (Manning, 2020).

Let's start with `OrderRepository`. Using the conventions you learned in chapter 5, define a method to find all orders created by the specified user. Spring Data will generate an implementation for it at runtime.

Listing 12.30 Defining a method returning orders created by a user

```
public interface OrderRepository
  extends ReactiveCrudRepository<Order,Long>
{
  Flux<Order> findAllByCreatedBy(String userId);     ❶
}
```

❶ Custom method to query only the orders created by the given user

Next we need to update the `getAllOrders()` method in `OrderService` to accept a username as input and use the new query method provided by `OrderRepository`.

Listing 12.31 Returning orders only for the specified user

```
@Service
public class OrderService {
  private final OrderRepository orderRepository;

  public Flux<Order> getAllOrders(String userId) {       ❶
    return orderRepository.findAllByCreatedBy(userId);
  }

  ...
}
```

❶ When requesting all orders, the response includes only those belonging to the given user.